

Inheritance

Lập trình hướng đối tượng

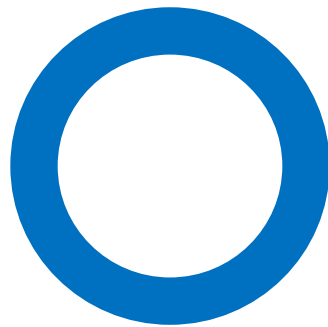
Hồ Tuấn Thanh

htthanh@fit.hcmus.edu.vn

Nội dung

- ❑ Khái niệm kế thừa
- ❑ Tầng vực trong kế thừa
- ❑ Định nghĩa lại hàm
- ❑ IS-A vs HAS-A

Khái niệm kế thừa



Ba tính chất của LT HĐT

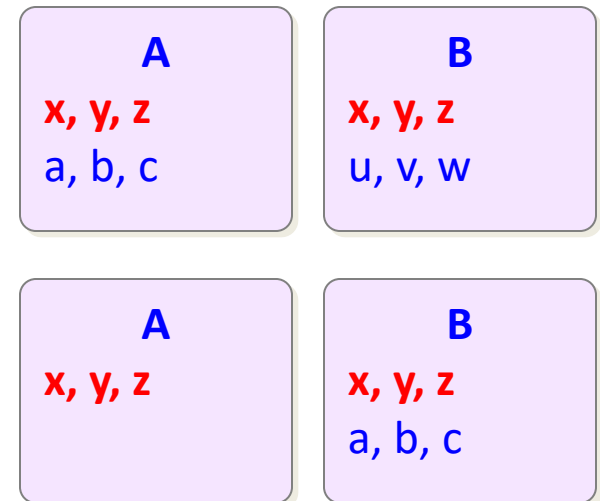
- ☐ Tính đóng gói, bao bọc
- ☐ Tính kế thừa
- ☐ Tính đa hình

Tính kế thừa

- “In object-oriented programming, inheritance is the mechanism of basing an object or class upon another object (prototypical inheritance) or class (class-based inheritance), retaining similar implementation.”
(Wikipedia)

Vấn đề trùng lặp thông tin

- ❑ Nhiều lớp có thông tin giống nhau
- ❑ Có 2 dạng trùng lặp thông tin:
 - Dạng chia sẻ: $A \cap B \neq \emptyset$
 - Dạng mở rộng: $B = A + \varepsilon$
- ❑ Nhược điểm:
 - Xây dựng tốn kém
 - Dung lượng lưu trữ lớn
 - Thay đổi phần chung khó khăn
- ❑ Giải quyết: **TÁI SỬ DỤNG**

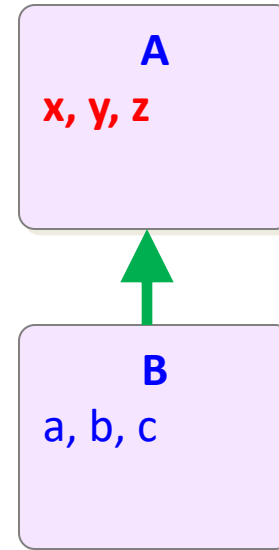
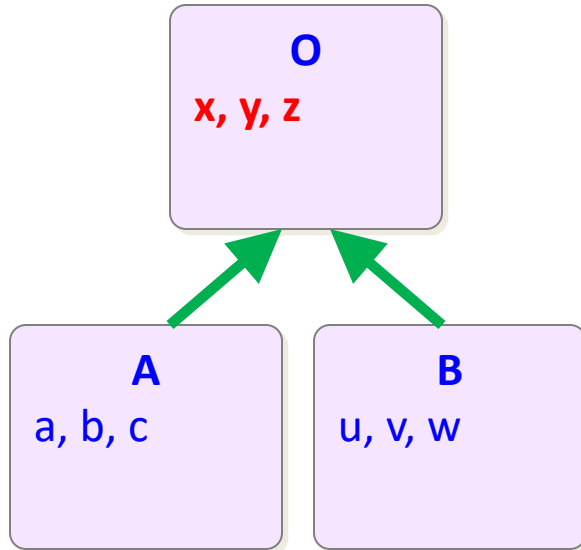


Khái niệm kế thừa

- ❑ Giải quyết vấn đề trùng lặp thông tin
- ❑ Cách thực hiện:
 - Định nghĩa lớp mới (lớp O) chứa các thuộc tính chung
 - Các lớp hiện tại kế thừa lớp mới
 - Khai báo những thuộc tính riêng trong lớp hiện tại
- ❑ Lớp mới (O) được gọi là lớp cơ sở, lớp cha
- ❑ Lớp hiện tại (A, B) được gọi là lớp kế thừa, lớp dẫn xuất, lớp con

Khái niệm kế thừa

- Nguyên tắc: **LỚP KẾ THỪA THỪA HƯỞNG TẤT CẢ TỪ LỚP CƠ SỞ**

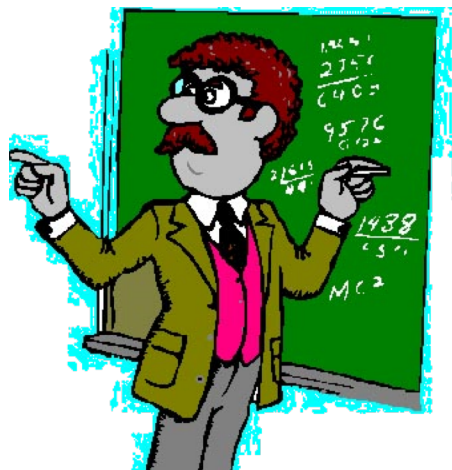


Khai báo trong C++

- ❑ class <Lớp kế thừa> : <Loại kế thừa> <Lớp cơ sở>
- ❑ Loại kế thừa: **public**, protected, private
- ❑ Ví dụ:

```
class A: public O
{
private:
    // Khai báo những thuộc tính mà chỉ A có
public:
    // Khai báo những hàm mà chỉ A có
};
```

Ví dụ



Thông tin:
Họ tên.
Mức lương.
Số ngày nghỉ.
Công việc:
Giảng dạy.
Tính lương.

Thông tin:
Họ tên.
Mức lương.
Số ngày nghỉ.
Lớp chủ nhiệm.

Công việc:
Giảng dạy.
Tính lương.
Sinh hoạt chủ nhiệm.



Ví dụ

□ Lớp GiaoVien & GVCN

```
class GiaoVien
{
    private:
        string hoTen;
        float mucLuong;
        int soNgayNghỉ;
    public:
        GiaoVien(string ht,
            float ml,
            int snn);
        void giangDay();
        float tinhLuong();
};
```

Các thuộc tính chung

Các hàm chung

```
class GVCN: public GiaoVien
{
    private:
        string lopCN;
    public:
        GVCN(string ht,
            float ml,
            int snn,
            string lcn);
        void sinhHoatCN();
};
```

Các thuộc tính riêng

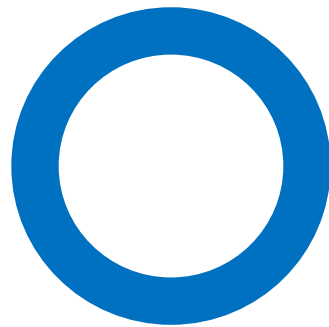
Các hàm riêng

Ví dụ

❑ Hàm main

```
void main()  
{  
    GiaoVien gv1("HTThanh",2000000,2);  
    gv1.giangDay();  
    cout<<gv1.tinhLuong();  
  
    GVCN gv2("LHMinh",3000000,3,"12CK2");  
    gv2.giangDay();  
    gv2.sinhHoatCN();  
    cout<<gv2.tinhLuong();  
}
```

Tâm vực trong kế thừa

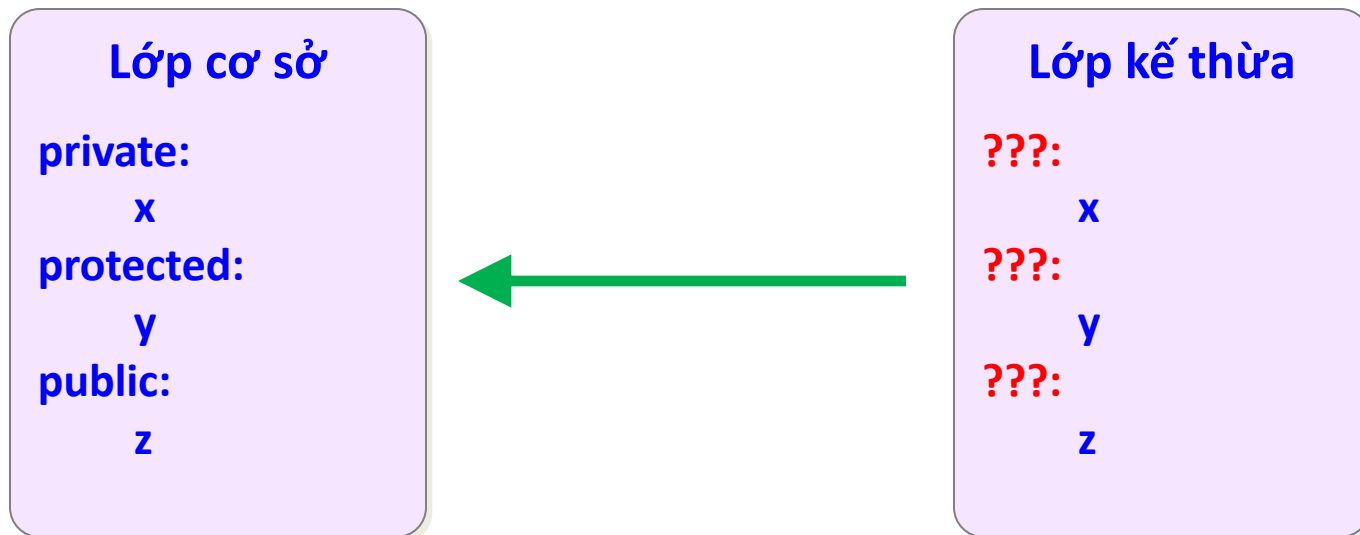


Lưu ý

- ❑ 3 loại kế thừa: **public**, protected, private
 - Đại đa số trường hợp sử dụng public
- ❑ 3 loại tầm vực: public, **protected**, private
 - Public: trong lớp truy xuất được, lớp con truy xuất được, ngoài lớp truy xuất được
 - Protected: trong lớp truy xuất được, lớp con truy xuất được, ngoài lớp không truy xuất được
 - Private: trong lớp truy xuất được, lớp con không truy xuất được, ngoài lớp không truy xuất được

Tầm vực trong kế thừa

- ❑ Vấn đề: Một thuộc tính/hàm đang ở tầm vực private/protected/public ở lớp cha thì ở lớp con, tầm vực của thuộc tính/hàm đó là gì?
- ❑ Trả lời: Do loại kế thừa quyết định



Bảng tầm vực kế thừa

Tầm vực	Kế thừa public	Kế thừa protected	Kế thừa private
public	public	protected	private
protected	protected	protected	private
private	Không truy xuất được	Không truy xuất được	Không truy xuất được

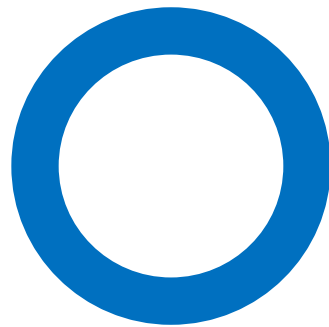
Bài tập

```
class A
{
private:
    int xA;
protected:
    int yA;
public:
    int zA;
    void fA()
    {
        cout<<xA<<yA<<zA<<endl;
    }
};
```

```
class B: public A
{
private:
    int xB;
protected:
    int yB;
public:
    int zB;
    void fB()
    {
        cout<<xA<<yA<<zA<<endl;
        cout<<xB<<yB<<zB<<endl;
    }
};
```

```
void main()
{
    A a;
    B b;
    cout<<a.xA<<a.yA<<a.zA<<endl;
    cout<<a.xB<<a.yB<<a.zB<<endl;
    cout<<b.xA<<b.yA<<b.zA<<endl;
    cout<<b.xB<<b.yB<<b.zB<<endl;
}
```

Định nghĩa lại hàm



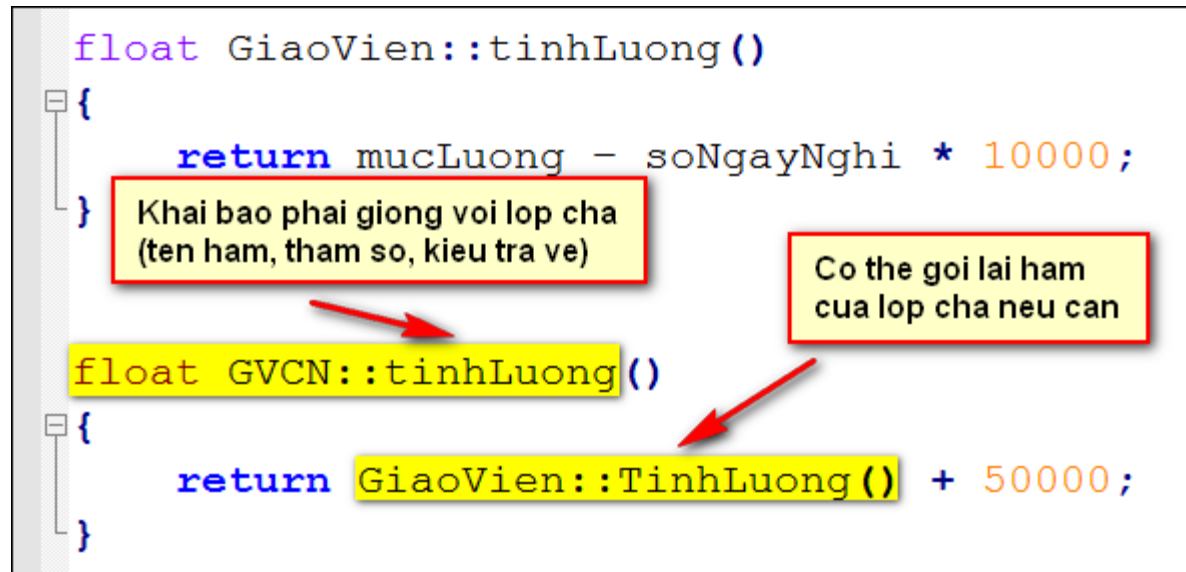
Định nghĩa lại hàm

- ❑ Nguyên tắc: **LỚP KẾ THỪA THỪA HƯỞNG TẤT CẢ TỪ LỚP CƠ SỞ**
- ❑ Như vậy: lớp cha có hàm $f \rightarrow$ lớp con có hàm f
- ❑ Lớp con muốn cách xử lí của hàm f khác với cách xử lí hàm f của lớp cha
- ❑ Lớp con có quyền định nghĩa lại (override) hàm f

Ví dụ

- ❑ GVCN kế thừa từ GiaoVien.
- ❑ GVCN tính lương khác GiaoVien.
 - $\text{Lương GV} = \text{Mức lương} - \text{Số ngày nghỉ} * 10000.$
 - $\text{Lương GVCN} = \text{Lương GV} + \text{Phụ cấp } 50000.$
- ❑ Trong lớp GVCN, ta có thể định nghĩa lại hàm `tinhLuong()`

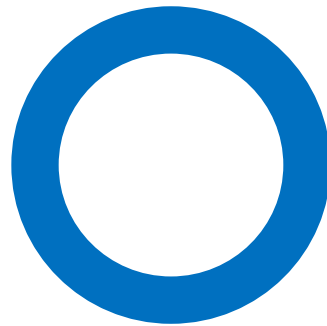
Ví dụ



Lưu ý

- ❑ Khi định nghĩa lại một hàm ở lớp con, cần khai báo hàm đó ở lớp con
- ❑ Khai báo hàm đó của lớp con **PHẢI GIỐNG Y HỆT** khai báo hàm đó của lớp cha
- ❑ Có thể gọi lại hàm của lớp cha nếu cần
 - Cú pháp **TenLopCha::TenHam(danh sách tham số)**

IS-A vs HAS-A

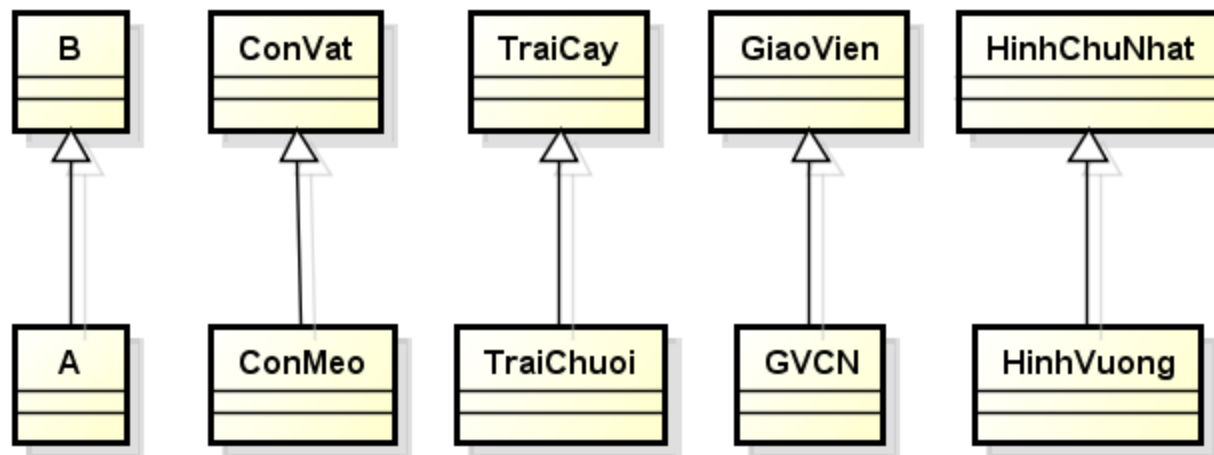


Quan hệ IS-A

- ❑ Lớp A có quan hệ IS-A với lớp B
 - A là một trường hợp đặc biệt của B
 - A cùng loại với B
 - A kế thừa B
- ❑ Ví dụ:
 - ConMeo là một loại ConVat
 - TraiChuoi là một loại TraiCay
 - GVCN là một loại GiaoVien
 - HìnhVuong là một loại HìnhChuNhat

Quan hệ IS-A

- ❑ Nguyên tắc: trước khi để lớp A kế thừa lớp B, cần tự đặt câu hỏi: **A có phải là một loại của B hay không?**
- ❑ Kí hiệu IS-A: mũi tên, đầu **tam giác** chỉ vào lớp cha



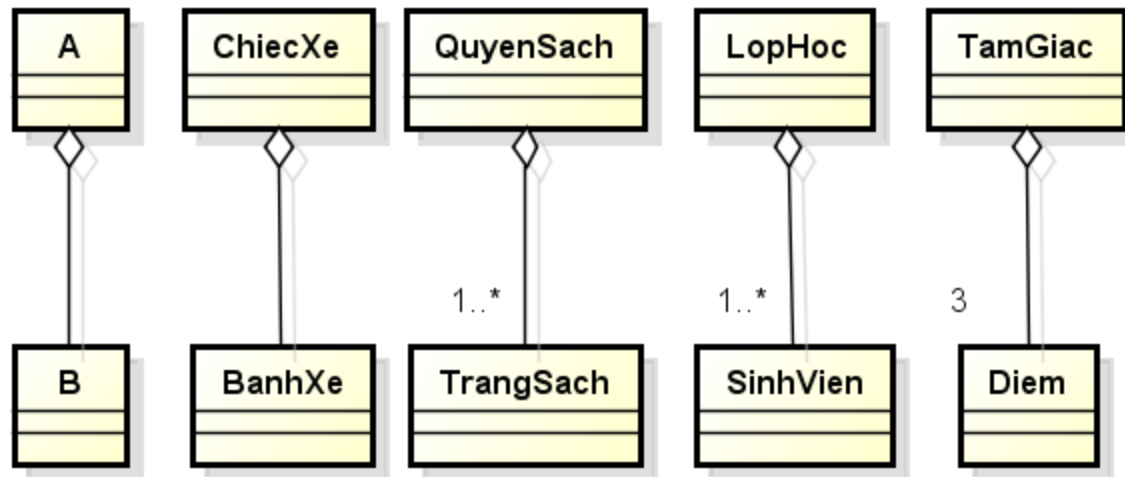
powered by Astah

Quan hệ HAS-A

- ❑ Lớp A có quan hệ HAS-A với lớp B
 - A bao hàm B
 - A chứa B
 - B là một bộ phận của A
 - Trong lớp A, ta có khai báo **thuộc tính** kiểu là lớp B
- ❑ Ví dụ:
 - ChiecXe có nhiều BanhXe
 - QuyenSach có nhiều TrangSach
 - LopHoc có nhiều SinhVien
 - TamGiac có 3 Diem

Quan hệ HAS-A

- ❑ Kí hiệu HAS-A: mũi tên, đầu **hình thoi** chỉ vào lớp chứa



powered by Astah

